
Viscous Effects

Ephraim Aung

Student ID: 920074048

Project #: 5

Date:11/30/2024

EAE 127: Applied Aerodynamics

University of California, Davis

Setup

In [203...

```
# Standard Imports
import math
import numpy as np
from scipy import integrate
import matplotlib.pyplot as plt
# others
import pandas as pd
import os
import pyxfoil
import mses
import vivi as vi
# embed the figures into the notebook
%matplotlib inline

### JUPYTER NOTEBOOK SETTINGS #####
# Plot all figures in full-size, no scroll bars
# %matplotlib inline
# Disable Python Warning Output

"""COMMENT OUT WHEN DEBUGGING"""
import warnings
warnings.filterwarnings('ignore')

### PLOTTING DEFAULTS BOILERPLATE (OPTIONAL)
# SET DEFAULT FIGURE APPEARANCE
import seaborn as sns # Fancy plotting package
# No Background fill, Legend font scale, frame on Legend (not sure what this does)
sns.set(style='whitegrid', font_scale=1.5, rc={'legend.frameon': True})
```

```

# Mark ticks on all four sides (overriding 'whitegrid')
sns.set_style('ticks')
# Ticks point inward
sns.set_style({'xticks.direction': 'in', 'ytick.direction': 'in'})
# Fixed invisible marker bug
sns.set_context(rc={'line.markeredgewidth': 0.1})
# restore default matplotlib colormap
mplcolors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd',
             '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf']
sns.set_palette(mplcolors)
# Get color cycle for manual colors
colors = sns.color_palette()
# Set matplotlib defaults (call after seaborn, which changes some defaults)
params={
    # FONT SIZES
    'axes.labelsize':30, # Axis Labels
    'axes.titlesize':30, # Title'font.size':28,# Textbox
    'xtick.labelsize':22, # Axis tick Labels
    'ytick.labelsize':22, # Axis tick Labels
    'legend.fontsize':24, # Legend font size
    'font.family':'serif',
    'font.fantasy':'xkcd',
    'font.sans-serif':'Helvetica',
    'font.monospace':'Courier',
    # AXIS PROPERTIES
    'axes.titlepad':2*6.0, # title spacing from axis
    'axes.grid':True, # grid on plot
    'figure.figsize':(8,8), # square plots
    'savefig.bbox':'tight', # reduce whitespace in saved figures
    # LEGEND PROPERTIES
    'legend.framealpha':0.5,
    'legend.fancybox':True,
    'legend.frameon':True,
    'legend.numpoints':1,
    'legend.scatterpoints':1,
    'legend.borderpad':0.1,
    'legend.borderaxespad':0.1,
    'legend.handletextpad':0.2,
    'legend.handlelength':1.0,
    'legend.labelspacing':0,
}

import matplotlib
matplotlib.rcParams.update(params) # update matplotlib defaults, call after seaborn
### end of boilerplate #####

colors = sns.color_palette() # color cycle

# folder to save results in
savedir = 'results'
    # make save folder
    # os.mkdir(savedir)

# Make Prints Look nice
class color:
    PURPLE = '\033[95m'

```

```

CYAN = '\033[96m'
DARKCYAN = '\033[36m'
BLUE = '\033[94m'
GREEN = '\033[92m'
YELLOW = '\033[93m'
RED = '\033[91m'
BOLD = '\033[1m'
UNDERLINE = '\033[4m'
END = '\033[0m'

```

1.1

In [204...

```

# Constants
antenna_width = 6 / 39.37 # 6 inches to meters
critical_Re = 5e5

# Sea Level properties (Taxiing)
rho_taxi = 1.225 # kg/m^3
mu_taxi = 1.7894e-5 # N·s/m^2
velocity_taxi = 12 * 0.51444 # knots to m/s

# Cruise properties at 35,000 ft
rho_cruise = 0.38428168 # kg/m^3
mu_cruise = 1.7894e-5 # N·s/m^2
velocity_cruise = 600 * 0.51444 # knots to m/s

# Compute Reynolds numbers
Re_taxi = rho_taxi * velocity_taxi * antenna_width / mu_taxi
Re_cruise = rho_cruise * velocity_cruise * antenna_width / mu_cruise

# Print results
print(f"Taxiing Reynolds Number: {Re_taxi:.2e}")
print(f"Cruise Reynolds Number: {Re_cruise:.2e}")

```

Taxiing Reynolds Number: 6.44e+04

Cruise Reynolds Number: 1.01e+06

For taxiing at sea level, the assumption of incompressible flow is valid since the flow velocity (12 knots or ~6.17 m/s) is much lower than the speed of sound, leading to negligible compressibility effects. However, for cruise conditions at 35,000 ft with a velocity of 600 knots (~308 m/s), the flow is approaching transonic speeds (Mach ~0.9). Compressibility effects, such as changes in density and pressure distribution, become significant in this regime, making the assumption of incompressibility less valid. A more accurate analysis at cruise should account for compressibility using compressible flow equations.

1.2

In [205...

```

import numpy as np
import matplotlib.pyplot as plt

# Constants

```

```

x = 1.0 # Example distance in meters
antenna_width = 6 / 39.37 # 6 inches to meters
u_taxi = 12 * 0.51444 # Taxiing
u_cruise = 600 * 0.51444 # Cruise

# Conversion factors
m_to_in = 39.37 # meters to inches
mps_to_knots = 1 / 0.51444 # meters per second to knots

# Reynolds numbers
Re_taxi = rho_taxi * u_taxi * antenna_width / mu_taxi
Re_cruise = rho_cruise * u_cruise * antenna_width / mu_cruise

# Boundary layer thickness calculations
delta_lam_taxi = 5.0 * x / np.sqrt(Re_taxi)
delta_turb_cruise = 0.16 * x / (Re_cruise**(1/7))

# Generate y/δ (non-dimensional) and corresponding u/u_e
y_non = np.linspace(0, 1, 100)
u_lam_non = 2 * y_non - y_non**2
u_turb_non = y_non**(1/7)

# Generate y (dimensional) and corresponding u
y_dim_taxi_lam = y_non * delta_lam_taxi * m_to_in # Convert to inches
y_dim_cruise_turb = y_non * delta_turb_cruise * m_to_in # Convert to inches

u_dim_taxi_lam = u_lam_non * u_taxi * mps_to_knots # Convert to knots
u_dim_cruise_turb = u_turb_non * u_cruise * mps_to_knots # Convert to knots

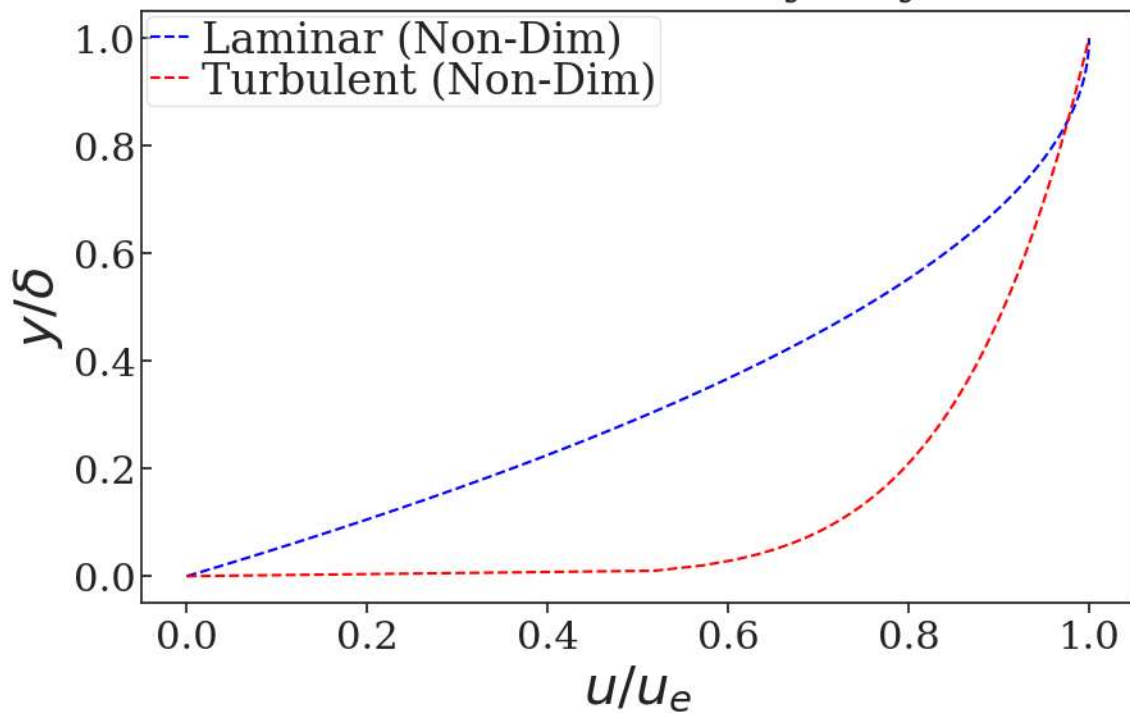
# Plot 1: Non-Dimensional Profiles
plt.figure(figsize=(10, 6))
plt.plot(u_lam_non, y_non, label="Laminar (Non-Dim)", color="blue", linestyle="--")
plt.plot(u_turb_non, y_non, label="Turbulent (Non-Dim)", color="red", linestyle="--")
plt.title("Non-Dimensional Boundary Layer Profiles")
plt.xlabel(r"$u / u_e$")
plt.ylabel(r"$y / \delta$")
plt.legend()
plt.grid()

# Plot 2: Dimensional Profiles
plt.figure(figsize=(10, 6))
plt.plot(u_dim_taxi_lam, y_dim_taxi_lam, label="Laminar", color="green", linestyle="solid")
plt.plot(u_dim_cruise_turb, y_dim_cruise_turb, label="Turbulent", color="orange", linestyle="solid")
plt.title("Dimensional Boundary Layer Profiles")
plt.xlabel("u (knots)")
plt.ylabel("y (inches)")
plt.legend()
plt.grid()

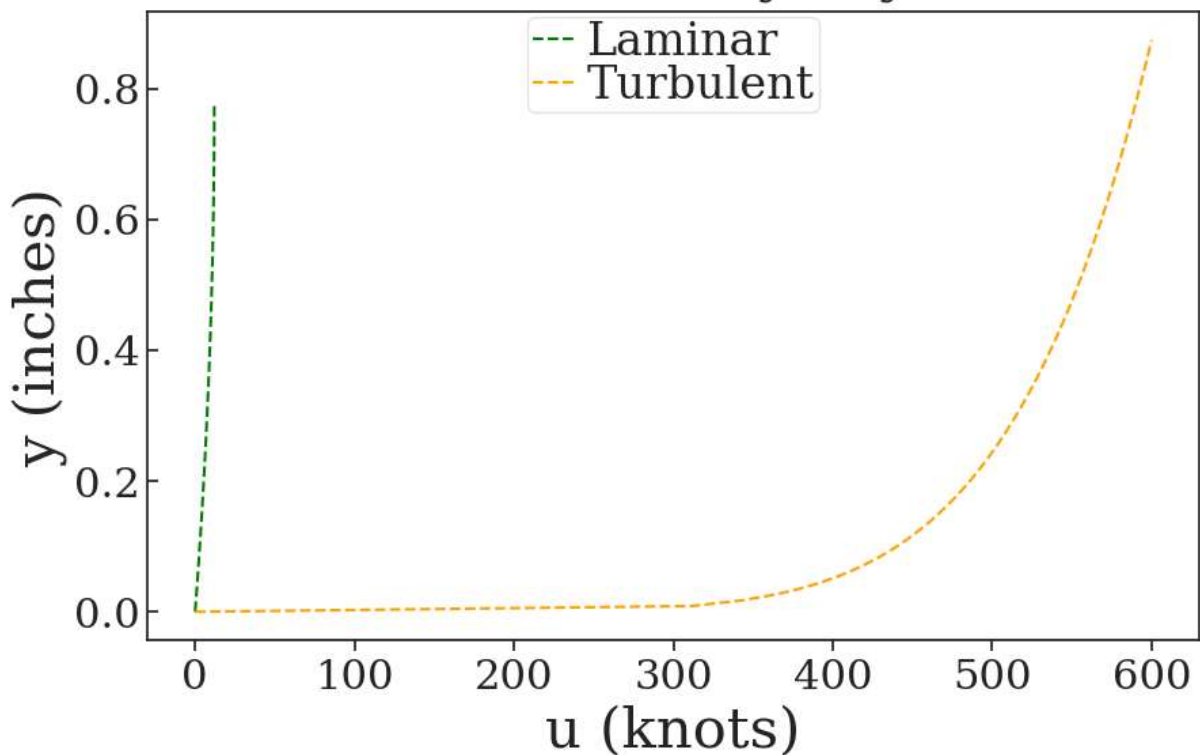
plt.show()

```

Non-Dimensional Boundary Layer Profiles



Dimensional Boundary Layer Profiles



The turbulent boundary layer has a fuller velocity profile close to the surface, which is evident from the higher velocities at lower (y/δ) in the plots. This makes it more resistant to adverse pressure gradients because the higher momentum near the wall helps the flow overcome deceleration and prevents separation. On the other hand, the laminar boundary layer, with slower velocities near the surface, is more likely to separate under similar

conditions. The turbulent boundary layer creates more skin friction due to increased mixing and steeper velocity gradients near the surface, leading to higher drag compared to the smoother, less energetic laminar flow.

1.3

```
In [206... import numpy as np

# Constants
L = 10 / 39.37 # 10 inches to meters
width = 6 / 39.37 # 6 inches to meters
S = 2 * L * width # Total surface area (both sides)

# Reynolds numbers
Re_taxi = rho_taxi * u_taxi * L / mu_taxi
Re_cruise = rho_cruise * u_cruise * L / mu_cruise

# Friction coefficients
Cf_lam_taxi = 1.328 / np.sqrt(Re_taxi)
Cf_turb_cruise = 0.074 / (Re_cruise**(1/5))

# Total Cf (both sides)
Cf_total_lam_taxi = 2 * Cf_lam_taxi
Cf_total_turb_cruise = 2 * Cf_turb_cruise

# Drag forces in Newtons
Df_lam_taxi = 0.5 * rho_taxi * u_taxi**2 * S * Cf_total_lam_taxi
Df_turb_cruise = 0.5 * rho_cruise * u_cruise**2 * S * Cf_total_turb_cruise

# Convert drag forces to pounds-force
Df_lam_taxi_lbf = Df_lam_taxi * 0.224809
Df_turb_cruise_lbf = Df_turb_cruise * 0.224809

# Output results
print("Taxiing:")
print(f"Laminar Cf: {Cf_total_lam_taxi:.6f}")
print(f"Laminar Drag Force: {Df_lam_taxi_lbf:.3f} lbf")

print("\nCruise:")
print(f"Turbulent Cf: {Cf_total_turb_cruise:.6f}")
print(f"Turbulent Drag Force: {Df_turb_cruise_lbf:.3f} lbf")
```

Taxiing:
Laminar Cf: 0.008107
Laminar Drag Force: 0.003 lbf

Cruise:
Turbulent Cf: 0.008414
Turbulent Drag Force: 2.681 lbf

The friction drag force on the antenna fin is about 0.003 lbf during taxiing and 2.681 lbf during cruise. For taxiing, the drag force is negligible due to the low velocity and laminar flow, making it insignificant. However, during cruise, the drag force increases significantly due to the higher velocity and turbulent flow. While 2.681 lbf is small compared to the

overall drag on the aircraft, it can still play a role in total drag, especially when considering other small drag sources that may add up.

2.1

In [207...

```
import numpy as np
import matplotlib.pyplot as plt
import pyxfoil # Assuming pyxfoil is correctly set up in your environment
import mses # Assuming mses is correctly set up in your environment

# Constants and given values
chord_length = 7.0 # meters
velocity = 8.0 # m/s
rho = 0.7719239999999999 # air density at 15,000 ft (kg/m^3)
mu = 1.6421239999999997e-5 # dynamic viscosity at 15,000 ft (N·s/m^2)
nu = mu / rho # kinematic viscosity (m^2/s)
re_crit = 5e5 # critical Reynolds number for transition

# Step 1: Load airfoil geometry from file
foil = "23012"
naca = True
Re = 0
alf = 0
pyxfoil.GetPolar(foil, naca, alf, Re) # Generates the necessary data file
filename = "Data/naca23012/naca23012.dat" # Adjust if file location differs
x, z = np.loadtxt(filename, unpack=True, skiprows=1)

# Scale airfoil coordinates for the chord length
x = x * chord_length
z = z * chord_length

# Split the airfoil into upper and lower surfaces
x_upper, x_lower = mses.MsesSplit(x, x)
z_upper, z_lower = mses.MsesSplit(x, z)

# Interpolate coordinates for a smooth curve
num_points = 300
xnew = np.linspace(0, chord_length, num_points)
z_upper = np.interp(xnew, x_upper, z_upper)
z_lower = np.interp(xnew, x_lower, z_lower)

# Step 2: Calculate Reynolds number along the chord
Re_x = velocity * xnew / nu

# Step 3: Determine boundary layer thickness
delta = np.zeros_like(xnew)
for i in range(num_points):
    if Re_x[i] < re_crit:
        delta[i] = (5.0 * xnew[i]) / np.sqrt(Re_x[i]) # Laminar flow
    else:
        delta[i] = (0.16 * xnew[i]) / Re_x[i]**(1/7) # Turbulent flow

# Step 4: Adjust the airfoil for boundary layer thickness
z_upper_with_bl = z_upper + delta
z_lower_with_bl = z_lower - delta
```

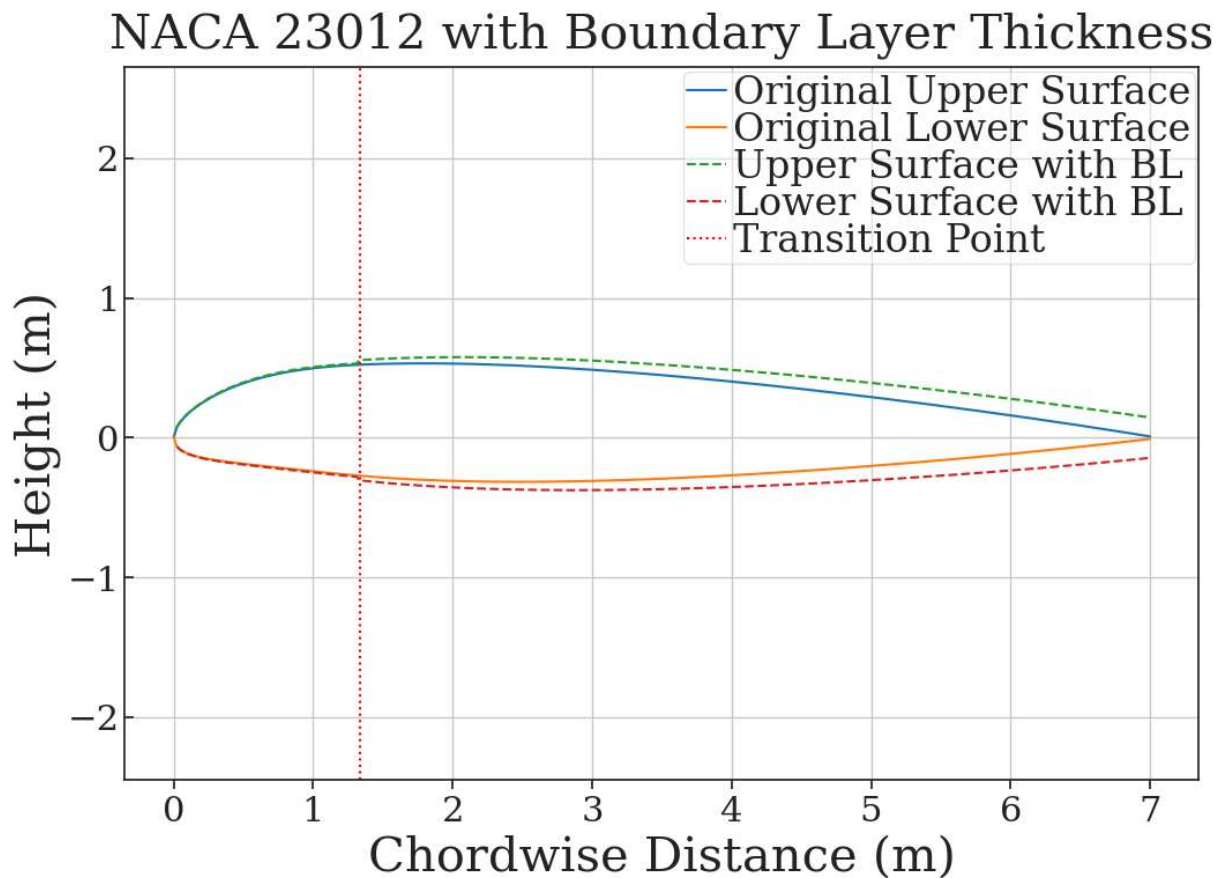


```

# Step 5: Plot original and adjusted airfoil geometry
transition_index = np.argmax(Re_x > re_crit) # Correct transition index
plt.figure(figsize=(12, 8))
plt.plot(xnew, z_upper, label="Original Upper Surface")
plt.plot(xnew, z_lower, label="Original Lower Surface")
plt.plot(xnew, z_upper_with_bl, label="Upper Surface with BL", linestyle="--")
plt.plot(xnew, z_lower_with_bl, label="Lower Surface with BL", linestyle="--")
plt.axvline(x=xnew[transition_index], color="red", linestyle=":", label="Transition")
plt.title("NACA 23012 with Boundary Layer Thickness")
plt.xlabel("Chordwise Distance (m)")
plt.ylabel("Height (m)")
plt.legend()
plt.grid(True)
plt.axis("equal")
plt.show()

# Step 6: Save results for submission
np.savetxt("airfoil_boundary_layer_results.csv",
           np.column_stack((xnew, z_upper, z_lower, delta)),
           delimiter=",",
           header="x,z_upper,z_lower,delta",
           comments="")

```



2.2

In [208...

```

import numpy as np
import matplotlib.pyplot as plt
import pyxfoil # Assuming pyxfoil is correctly set up in your environment

```



```

import mses # Assuming mses is correctly set up in your environment

# Constants and given values
chord_length = 7.0 # meters
velocity = 8.0 # m/s
rho = 0.7719239999999999 # Air density at 15,000 ft (kg/m^3)
mu = 1.642123999999997e-5 # Dynamic viscosity at 15,000 ft (N·s/m^2)
nu = mu / rho # Kinematic viscosity (m^2/s)
Re_crit = 5e5 # Critical Reynolds number for transition
angles_of_attack = [2, 8, 12] # Angles of attack in degrees

# Reynolds numbers for evaluation
Re_values = [1e5, Re_crit, 1e6]

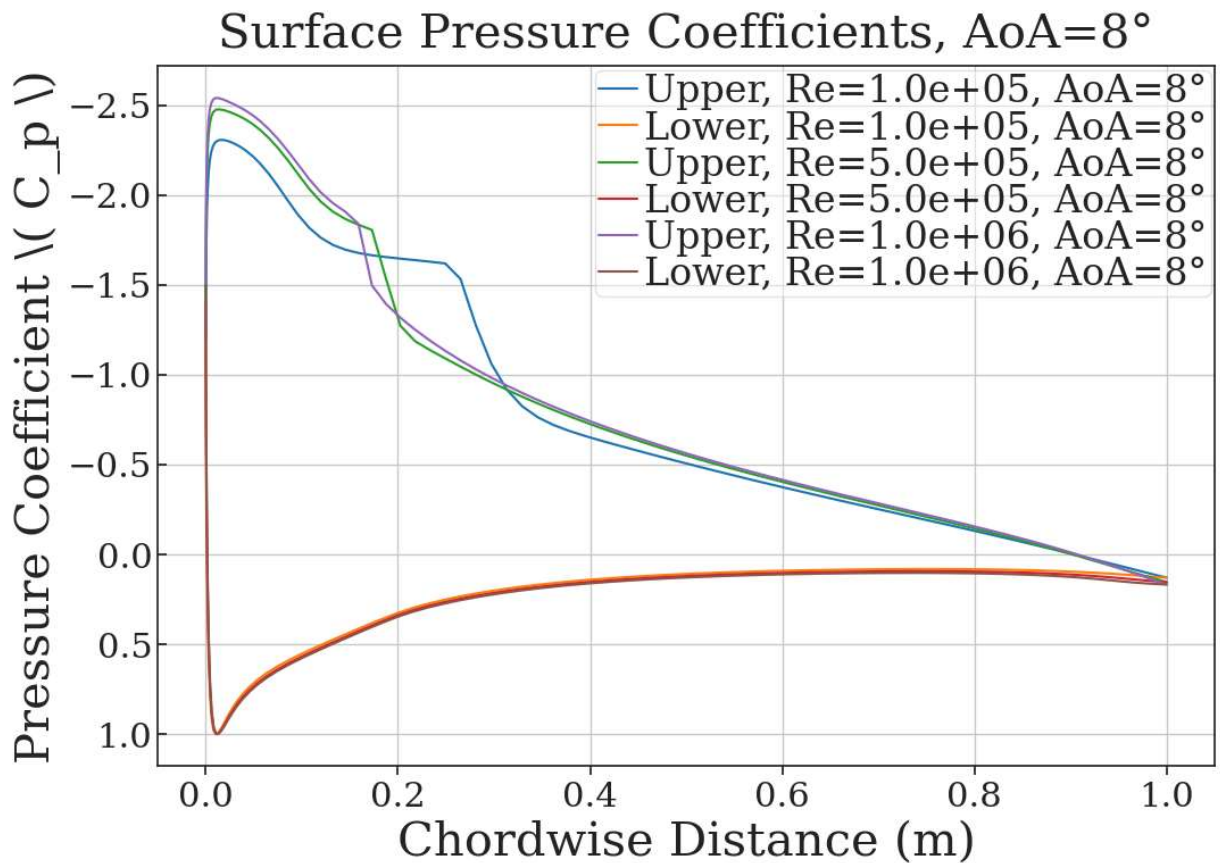
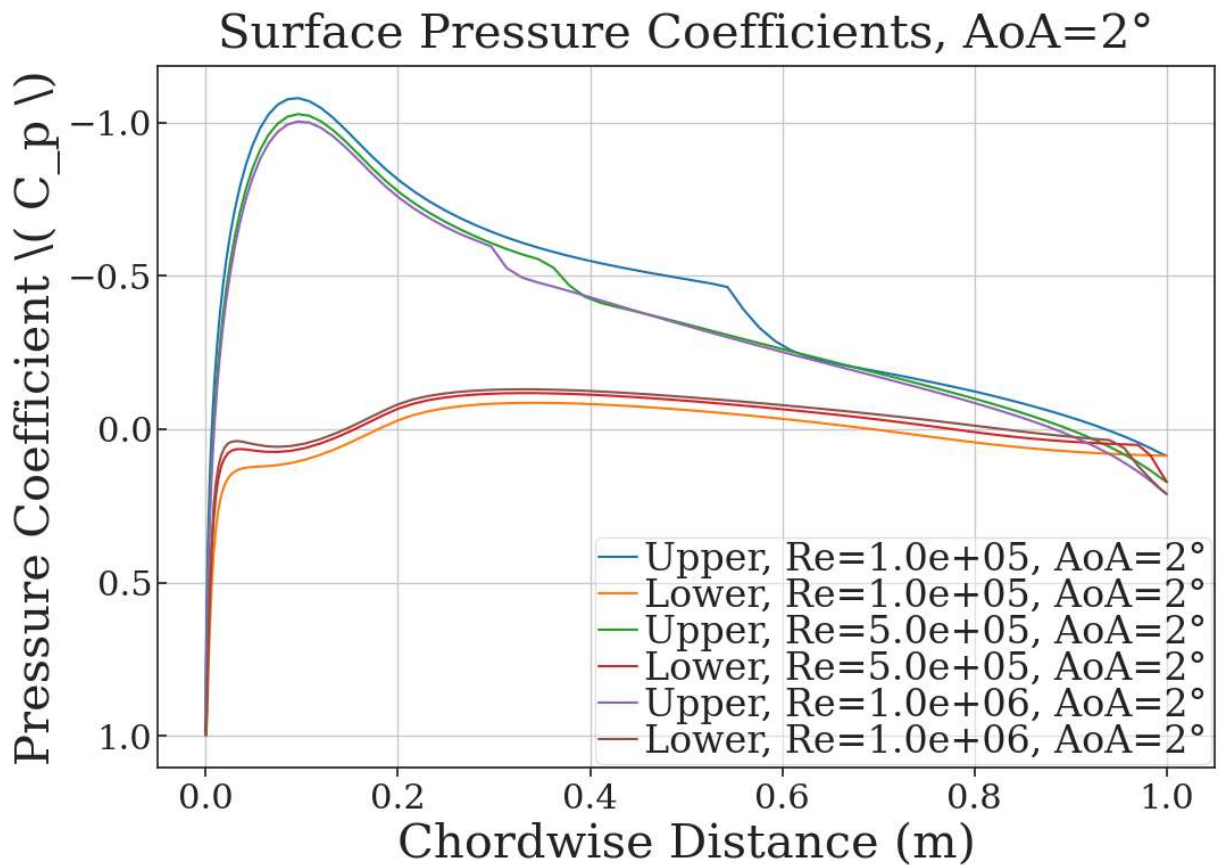
# Helper function to load surface pressure coefficient and geometry data
def load_airfoil_cp_data(foil, alf, Re):
    pyxfoil.GetPolar(foil, True, alf, Re) # Generate the required data
    filename = f"Data/naca23012/naca23012_surfCP_Re{Re:.2e}a{alf:.1f}.dat"
    x, y, Cp = np.loadtxt(filename, unpack=True, skiprows=3)
    x_upper, x_lower = mses.MsesSplit(x, x)
    Cp_upper, Cp_lower = mses.MsesSplit(x, Cp)
    return x_upper, x_lower, Cp_upper, Cp_lower

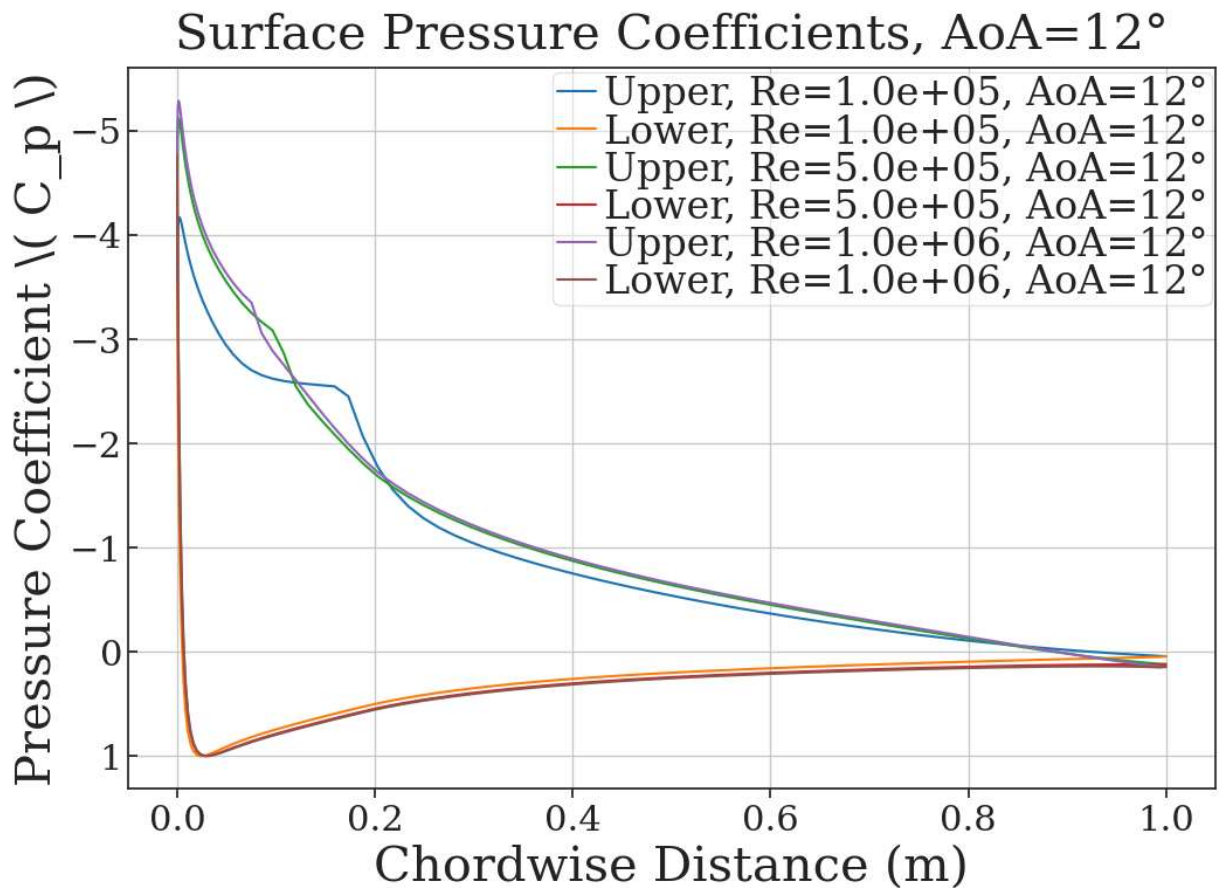
# Plotting for each angle of attack
for alf in angles_of_attack:
    plt.figure(figsize=(12, 8))
    for Re in Re_values:
        # Load data
        x_upper, x_lower, Cp_upper, Cp_lower = load_airfoil_cp_data("23012", alf, Re)

        # Plot upper and lower surface pressure coefficients
        plt.plot(x_upper, Cp_upper, label=f"Upper, Re={Re:.1e}, AoA={alf}°")
        plt.plot(x_lower, Cp_lower, label=f"Lower, Re={Re:.1e}, AoA={alf}°")

    # Finalize plot for the current angle of attack
    plt.title(f"Surface Pressure Coefficients, AoA={alf}°")
    plt.xlabel("Chordwise Distance (m)")
    plt.ylabel("Pressure Coefficient \ ( C_p \)")
    plt.gca().invert_yaxis() # Conventional for pressure plots
    plt.legend()
    plt.grid(True)
    plt.show()

```





3.0

Momentum thickness (θ) measures the reduction in flow momentum near the surface due to viscosity. Displacement thickness (δ^*) shows how much the flow is pushed away from the surface because of the slower velocity within the boundary layer. Boundary layer thickness (δ) is the distance from the surface to where the flow velocity nearly matches the free-stream velocity, showing how far the viscous effects extend. These characteristics describe how viscosity affects the flow near a surface.

3.1

In [209...

```
foil = '24012'
naca = True

# Reynold's number and AOA do not matter, only geometry is needed
Re = 0
alf = 0

pyxfoil.GetPolar(foil, naca, alf, Re)
filename = 'Data/naca24012/naca24012.dat'

x, z = np.loadtxt(filename, unpack = True, skiprows = 1)

# Plot Airfoil
plt.figure(figsize = (20,12))
plt.grid(True)
```

```

plt.title('NACA 24012')
plt.xlabel('x (m)', fontsize = 16)
plt.ylabel('y (m)', fontsize = 16)
#plt.plot(x, z, label = 'NACA 23012')
plt.axis('equal')

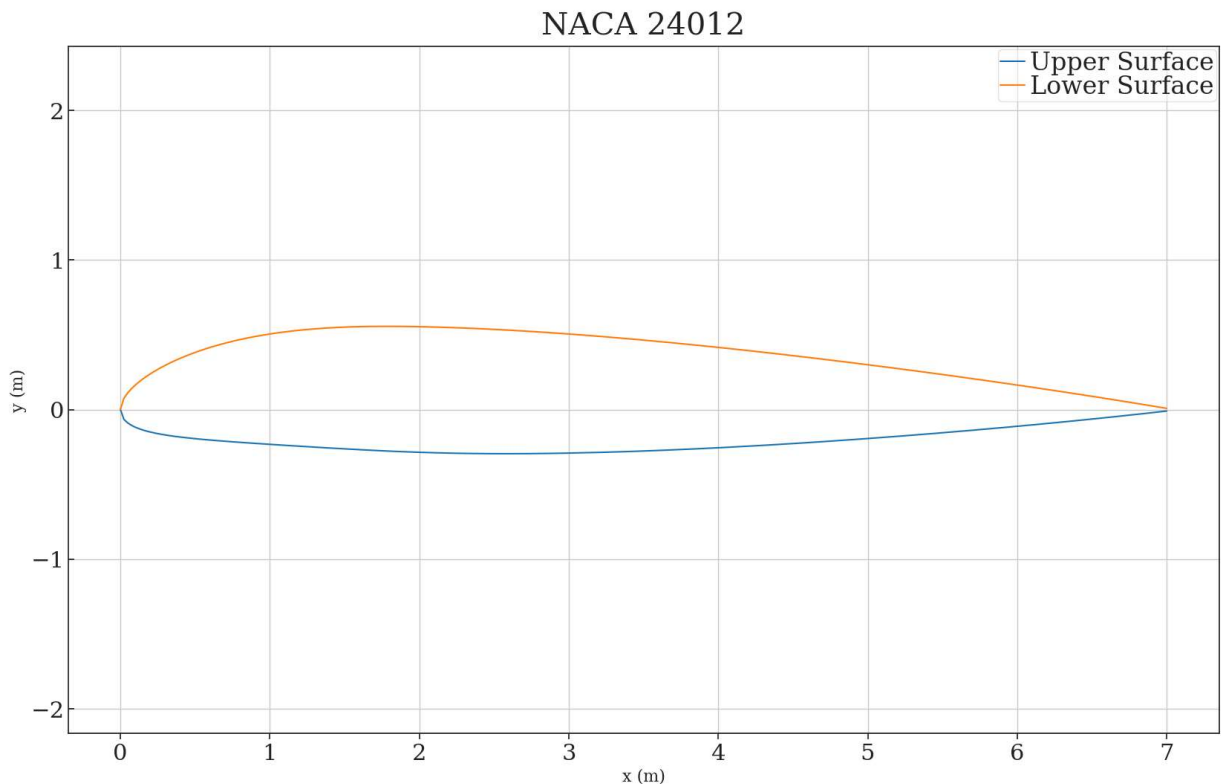
Re = 5 * (10**5)
alf = 12
pyxfoil.GetPolar(foil, naca, alf, Re)
filename = 'Data/naca24012/naca24012_surfCP_Re5.00e+05a12.0.dat'
x, y, Cp = np.loadtxt(filename, unpack = True, skiprows = 3)

# Splitting the airfoil coordinates
xUpper, xLower = mses.MsesSplit(x, x)
yUpper, yLower = mses.MsesSplit(x, y)
CpUp, CpLow = mses.MsesSplit(x, Cp)

# Interpolation to make sure values are the same size
xnew1 = np.linspace(0, 1, 300)
yLower = np.interp(xnew1, xLower, yLower)
yUpper = np.interp(xnew1, xUpper, yUpper)

# Plot 7x sized NACA 23012 Airfoil
plt.plot()
plt.plot(xnew1*7, yLower*7, label = 'Upper Surface')
plt.plot(xnew1*7, yUpper*7, label = 'Lower Surface')
plt.legend(loc = 'best')
plt.show()

```



In [210... name = 'naca24012'

```
# VIV iterations
```

```

iterations = [
    vi.VIvI(name, 0, niter = i, Vinf = 2, mu = 1.79E-5, rho = 1.225)
    for i in range(4)
]

# Define colors for each iteration
colors = ['k', 'r', 'g', 'b']
labels = ['Iteration 0', 'Iteration 1', 'Iteration 2', 'Iteration 3']

# Set up plot
width = 14
fig = plt.figure(figsize=(width, width / 4)) # Save figure object as variable
ax = fig.add_subplot(1, 1, 1) # Use axis object to modify figure
ax.set_title('VIvI Geometry Iterations (NACA 24012)')
plt.xlabel('x', fontsize=16)
plt.ylabel('y', fontsize=16)
plt.axis('equal')

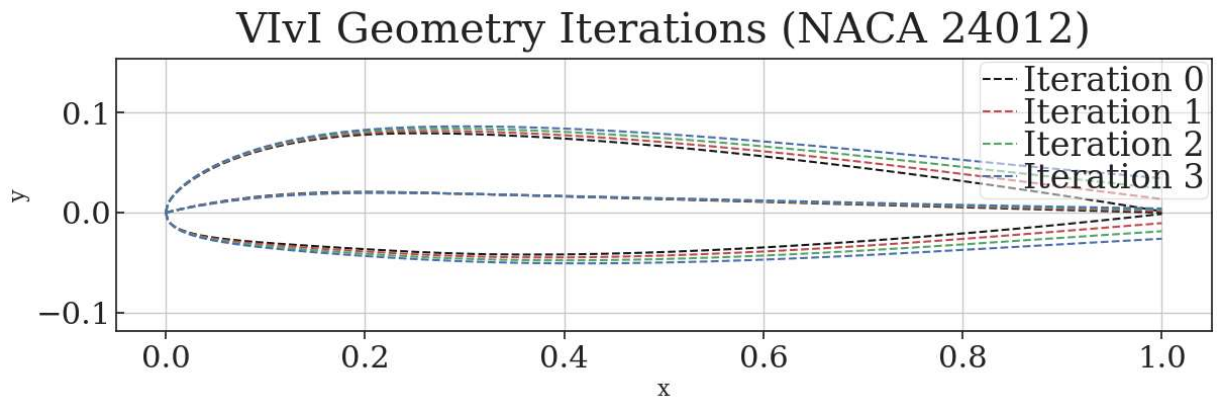
# Iterate through solutions, colors, and labels
for solution, color, label in zip(iterations, colors, labels):
    # Plot upper and lower surface geometry
    ax.plot(solution['up']['x'], solution['up']['z'], color = color, linestyle = '-')
    ax.plot(solution['lo']['x'], solution['lo']['z'], color = color, linestyle = '-')

    # Plot camber line with label
    ax.plot(
        solution['lo']['x'],
        (solution['up']['z'] + solution['lo']['z']) / 2,
        color = color,
        linestyle = '--',
        label = label
    )

# Add Legend and show the plot
plt.legend()
plt.show()

```

COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE



In [211...

```

name = 'naca24012'

# VivI iterations
iterations = [
    vi.VivI(name, 0, niter = i, Vinf = 2, mu = 1.79E-5, rho = 1.225)
    for i in range(4)
]

# Define colors and labels for each iteration
colors = ['k', 'r', 'g', 'b']
labels = ['Iteration 0', 'Iteration 1', 'Iteration 2', 'Iteration 3']

# Set up plot
fig = plt.figure(figsize=(width,width/4))
ax = fig.add_subplot(1, 1, 1)
ax.set_title('VivI Geometry Iterations (NACA24012) - Camber Lines Only')
plt.xlabel('x', fontsize=16)
plt.ylabel('y', fontsize=16)

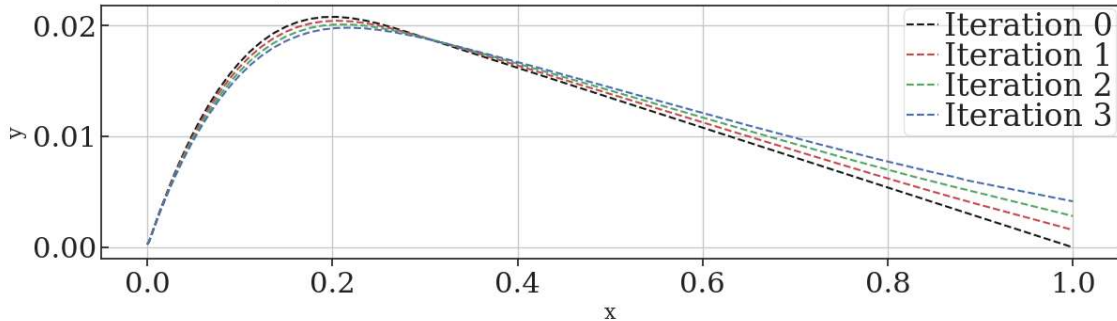
# Plot camber lines for each iteration
for solution, color, label in zip(iterations, colors, labels):
    ax.plot(
        solution['lo']['x'],
        (solution['up']['z'] + solution['lo']['z']) / 2,
        color=color,
        linestyle='--',
        label=label
    )

# Add Legend
plt.legend()
plt.show()

```


COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
 RETURNING ALL DATA, NOT FOR RELEASE

Vlvi Geometry Iterations (NACA24012) - Camber Lines Only



The Vlvi iterations show that displacement increases with each iteration, highlighting the growing impact of boundary layer effects as the viscous and inviscid solutions interact. Rather than converging, the geometry seems to diverge, with each iteration amplifying the displacement and thickening the boundary layer. This suggests that a pre-chosen iteration limit is necessary to avoid unrealistic changes. The increase in displacement demonstrates viscous decambering, where the boundary layer progressively alters the airfoil's camber line with each iteration.

3.2

```
In [212... import vivi
# Gather information for the first iteration
foil='24012'
naca = True
currentiter = 0
pyxfoil.GetPolar(foil, naca, alf, Re = 0)
# Parameters for Viscous/Inviscid function
foil = 'naca24012'
Vinf = 2
mu=1.79E-5
rho=1.225
naca= True
# Grab data theta0_up, ue0_up, theta0_lo, ue0_lo

Vinf=2
# Gather tau from Vlvi
dfs3 = vi.Vlvi(foil, alf, 3, Vinf, mu, rho)
tau_up = dfs3['up']['tau']
tau_lo = dfs3['lo']['tau']
```



```

# Calculate Skin Friction coefficient
Cf = (tau_up + tau_lo)/(0.5 * rho * Vinf**2)

# Calculate CD
xnew = np.linspace(0, 1, len(Cf)-2) # remove 2 terms since they are inf
Cd = np.trapz(Cf[2:],xnew)
print("Cd value for 3rd iteration = {}".format(Cd*2))

# Display values
dictionary = {
    'Type': ['Inviscid (Re=0) VIVI', 'Inviscid (Re=0) Xfoil', 'Viscid (Re=5E5) Xfoi
    'Cl': [0.0558, 0.1625, 1.3823],
    'Cd': [0.008322, 0, 0.01977]
}

df = pd.DataFrame(dictionary)
print(df)

```

```

COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
COMPUTING WALL SHEAR STRESS IN MAIN PROGRAM, NOT FOR RELEASE
RETURNING ALL DATA, NOT FOR RELEASE
Cd value for 3rd iteration = 0.008322567074973199
      Type      Cl      Cd
0  Inviscid (Re=0) VIVI  0.0558  0.008322
1  Inviscid (Re=0) Xfoil  0.1625  0.000000
2  Viscid (Re=5E5) Xfoil  1.3823  0.019770

```

The VIV solution initially shows much lower lift (Cl) and drag (Cd) than the XFOIL results. The Cl starts at 0.0558, far below XFOIL's inviscid Cl of 0.1625, and increases with each iteration, approaching the viscous XFOIL value of 1.3823. For drag, VIV starts with a Cd of 0.008322, higher than the inviscid XFOIL result (0.000000), but still lower than the viscous XFOIL Cd of 0.019770. While the VIV method improves over iterations, it doesn't match the accuracy of the full XFOIL model.

3.3

One limitation of the VIV tool is its use of the inviscid panel method, which discretizes elements on a surface and applies boundary conditions to create the shape. If there aren't enough panels, errors can occur, as the sum of all sources for each panel must be zero. If this condition isn't met, it implies the body is absorbing mass from the flow. Another limitation is that this method can't model rotational or supersonic flows because it assumes inviscid flow, which doesn't account for viscosity.